

Tutorial6-torch

November 11, 2025

Name: __HAN Zhuo__

EID: __zhuohan3__

1 CS5489 - Tutorial 6

1.1 Using Deep Features

In this tutorial you will train a classifier to classify images with different types of cakes using deep features.

First we need to initialize Python. Run the below cell.

```
[1]: %matplotlib inline
import matplotlib_inline # setup output image format
matplotlib_inline.backend_inline.set_matplotlib_formats('svg')
import matplotlib.pyplot as plt
import matplotlib
from numpy import *
from sklearn import *
import os
import zipfile
import fnmatch
import skimage.io
import skimage.transform
```

Second, initialize pytorch and its submodules

```
[2]: import torch
import logging
from torch import nn
from copy import deepcopy
logging.basicConfig()
import sys
from sklearn.preprocessing import LabelEncoder
from torchvision import models
import numpy as np
from torchvision import transforms
import torchvision.models as models
import torch.optim as optim
```

```

from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import ParameterGrid
import numpy as np
from tqdm import tqdm
from torch.optim.lr_scheduler import ReduceLROnPlateau
print("Python:", sys.version, "Torch:", torch.__version__)

```

Python: 3.12.12 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 20:16:04)
 [GCC 11.2.0] Torch: 2.9.0+cu130

1.2 Loading Data and Pre-processing

Next we need to load the images. Download `cakes.zip`, and put it in the same directory as this ipynb file. **Do not unzip the file.** Then run the following cell to load the images.

```

[3]: imgdata = []
    classes = []

    # load the zip file
    filename = 'cakes.zip'
    zfile = zipfile.ZipFile(filename, 'r')

    for name in zfile.namelist():
        # check file name matches
        if fnmatch.fnmatch(name, "cakes/*/*.jpg"):

            # filename is : cakes/class/file.jpg
            (fdir1, fname) = os.path.split(name) # get file name
            (fdir2, fclass) = os.path.split(fdir1) # get class

            # open file in memory, and parse as an image
            myfile = zfile.open(name)
            #img = matplotlib.image.imread(myfile)
            img = skimage.io.imread(myfile)
            img2 = skimage.transform.resize(img, (224,224), anti_aliasing=False)

            myfile.close()

            imgdata.append(img2)
            classes.append(fclass)

    zfile.close()
    print(len(imgdata))
    print(img.size)

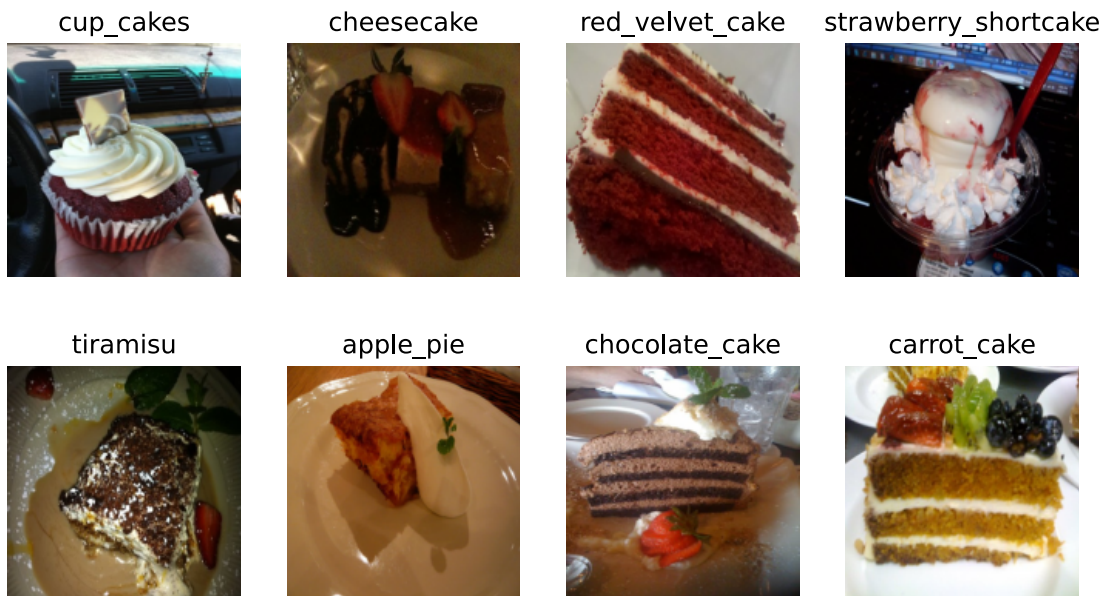
```

800
 589824

Each image is a 224x224. There are 8 classes of different cakes. Run the below code to show

examples of each class.

```
[4]: plt.figure(figsize=(9,5))
      for i in range(8):
          plt.subplot(2,4,i+1)
          ind = i*100
          plt.imshow(imgdata[ind])
          plt.title(classes[ind])
          plt.axis('off')
      plt.show()
```



Next, store the images into a Tensor. ResNet assumes the data is [0,255] range, so we need to scale the images that were read in with skimage.

```
[5]: # PyTorch version
      Xraw = torch.zeros((len(imgdata), 3, 224, 224))

      # Define preprocessing pipeline
      preprocess = transforms.Compose([
          transforms.ToPILImage(),
          transforms.Resize(224),
          transforms.ToTensor(),
          transforms.Lambda(lambda x: x * 255)
      ])

      for i, img in enumerate(imgdata):
          x = np.array(img)
```

```
x = preprocess(x)
Xraw[i] = x
```

Finally, we convert the class label strings into class numbers.

```
[6]: print("Class labels (strings):", np.unique(classes))
```

```
le = LabelEncoder()
Y = le.fit_transform(classes)
print("Converted labels:")
print(Y)

Y_tensor = torch.LongTensor(Y)
Yb = torch.zeros(len(Y), len(le.classes_))
Yb.scatter_(1, Y_tensor.unsqueeze(1), 1)

print("\nOne-hot encoded tensor:")
print(Yb)
print("Shape:", Yb.shape)
```

Class labels (strings): ['apple_pie' 'carrot_cake' 'cheesecake' 'chocolate_cake' 'cup_cakes']

```
'red_velvet_cake' 'strawberry_shortcake' 'tiramisu']
```

Converted labels:

[illegible]

One-hot encoded tensor:

```

tensor([[0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        [0., 0., 0., ..., 0., 0., 0.],
        ...,
        [0., 1., 0., ..., 0., 0., 0.],
        [0., 1., 0., ..., 0., 0., 0.],
        [0., 1., 0., ..., 0., 0., 0.]])
Shape: torch.Size([800, 8])

```

1.3 Extract Deep features

Now preprocess the images in `imdata` and put them into a 4D tensor for ResNet. Store it in a tensor `Xim`.

Note that `preprocess_input` may overwrite its input data, so you may need to make a copy if you want to preserve `Xraw`.

```
[7]: Xraw.shape
```

```
[7]: torch.Size([800, 3, 224, 224])
```

```

[8]: ### INSERT YOUR CODE HERE ###
Xim = Xraw.clone().float()
Xim = Xim / 255.0

imagenet_mean = torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1)
imagenet_std  = torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1)

Xim = (Xim - imagenet_mean) / imagenet_std

```

```
[9]: Xim.shape
```

```
[9]: torch.Size([800, 3, 224, 224])
```

Extract features from Resnet50, call it `Xf`.

```

[10]: ### INSERT YOUR CODE HERE ###
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
resnet = resnet.to(device)
resnet.eval()

feature_extractor = nn.Sequential(*list(resnet.children())[:-1]).to(device)
feature_extractor.eval()

N = Xim.size(0)
batch_size = 32
Xf = torch.zeros((N, 2048))

```

```

with torch.no_grad():
    for start in range(0, N, batch_size):
        end = start + batch_size
        if end > N:
            end = N
        xb = Xim[start:end].to(device)
        feats = feature_extractor(xb)
        feats = feats.view(feats.size(0), -1)    #
        Xf[start:end] = feats.cpu()

```

```

[11]: print(f"Xf shape: {Xf.shape}")
      print(f"Device: {device}")

```

```

Xf shape: torch.Size([800, 2048])
Device: cuda

```

1.4 Train a classifier

Split the data into training and testing for the classifier.

```

[12]: # randomly split data into train and test set
      ( trainXf, testXf,          # features
        trainY, testY,          # class labels
        trainYb, testYb,       # class one-hot vectors
        trainXim, testXim,     # processed images
        trainingdata, testingdata, # raw images
      ) = \
        model_selection.train_test_split(Xf, Y, Yb, Xim, imgdata,
        train_size=0.8, test_size=0.2, random_state=4487)

      print(trainXf.shape)
      print(testXf.shape)
      print(trainY.shape)
      print(testY.shape)
      # cleanup memory
      del trainingdata, Xraw

```

```

torch.Size([640, 2048])
torch.Size([160, 2048])
(640,)
(160,)

```

Now train a few traditional classifier (e.g, SVM, logistic regression, random forest, etc.) and compute the accuracy.

```

[13]: ### INSERT YOUR CODE HERE ###
      X_tr = trainXf.numpy()
      X_te = testXf.numpy()
      y_tr = trainY

```

```

y_te = testY

scaler = preprocessing.StandardScaler()
X_tr_std = scaler.fit_transform(X_tr)
X_te_std = scaler.transform(X_te)

```

```

[14]: # LR
logreg = linear_model.LogisticRegression(
    max_iter=2000,
    n_jobs=-1,
    multi_class="auto"
)
logreg.fit(X_tr_std, y_tr)
pred_lr = logreg.predict(X_te_std)
acc_lr = metrics.accuracy_score(y_te, pred_lr)
print(f"[LogReg] accuracy = {acc_lr:.4f}")

```

/home/hanzhuo/miniconda3/envs/hz/lib/python3.12/site-packages/sklearn/linear_model/_logistic.py:1272: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.8. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

```
warnings.warn(
```

[LogReg] accuracy = 0.5750

```

[15]: # SVM
from sklearn.svm import LinearSVC

this_svm = LinearSVC()
this_svm.fit(X_tr_std, y_tr)
pred_svm = this_svm.predict(X_te_std)
acc_svm = metrics.accuracy_score(y_te, pred_svm)
print(f"[LinearSVM] accuracy = {acc_svm:.4f}")

```

[LinearSVM] accuracy = 0.5437

```

[16]: print("Classification report (LinearSVM):")
print(metrics.classification_report(y_te, pred_svm, target_names=[str(c) for c_
    ↪in range(len(set(y_tr)))]))

```

Classification report (LinearSVM):

	precision	recall	f1-score	support
0	0.74	0.81	0.77	21
1	0.40	0.59	0.48	17
2	0.39	0.39	0.39	18
3	0.40	0.26	0.32	23
4	0.71	0.91	0.80	22
5	0.50	0.47	0.48	17

6	0.61	0.64	0.62	22
7	0.42	0.25	0.31	20
accuracy			0.54	160
macro avg	0.52	0.54	0.52	160
weighted avg	0.53	0.54	0.53	160

1.5 Analyze results

Run the below code to visualize the predictions on the test set. `mypred` are the class predictions from your classifier.

```
[17]: # get the class labels
predYcl = le.inverse_transform(pred_svm)
testYcl = le.inverse_transform(testY)

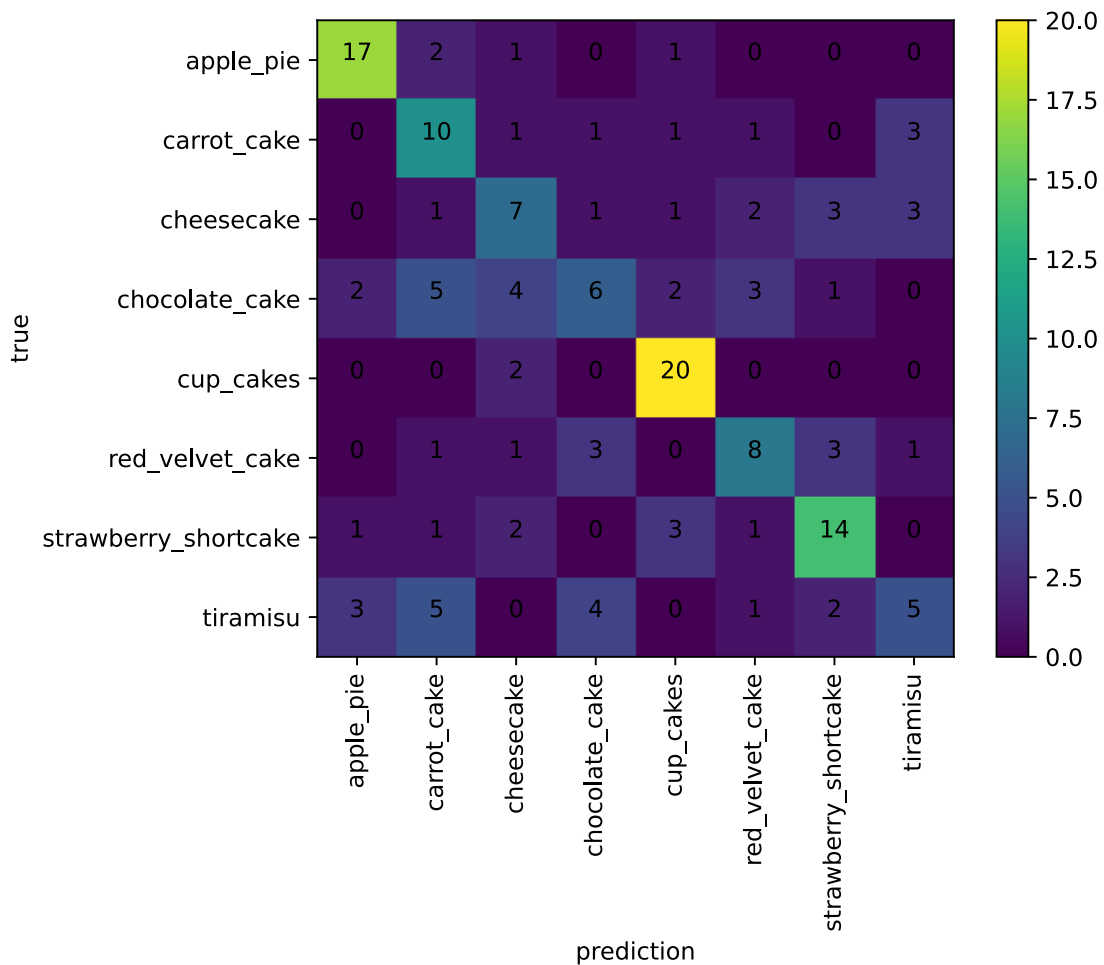
plt.figure(figsize=(10,50))
px = 8
py = int(ceil(len(predYcl)/5))
for i in range(len(predYcl)):
    plt.subplot(py,px,i+1)
    mytitle = predYcl[i] + "\n(" + testYcl[i] + ")"
    plt.imshow(testimgdata[i], interpolation='nearest')
    plt.title(mytitle, size=8)
    plt.gca().set_axis_off()
plt.show()
```



```
[18]: CM = metrics.confusion_matrix(testY, pred_svm)
```

```
plt.imshow(CM, interpolation='nearest')
plt.colorbar()
```

```
tick_marks = arange(len(le.classes_))
plt.xticks(tick_marks, le.classes_, rotation=90)
plt.yticks(tick_marks, le.classes_)
plt.xlabel('prediction')
plt.ylabel('true')
for i in range(len(le.classes_)):
    for j in range(len(le.classes_)):
        plt.text(j,i,"{}".format(CM[i,j]), horizontalalignment='center')
plt.show()
```



```
[19]: # get the class labels
predYcl = le.inverse_transform(pred_lr)
testYcl = le.inverse_transform(testY)

plt.figure(figsize=(10,50))
px = 8
py = int(ceil(len(predYcl)/5))
for i in range(len(predYcl)):
    plt.subplot(py,px,i+1)
    mytitle = predYcl[i] + "\n(" + testYcl[i] + ")"
    plt.imshow(testimgdata[i], interpolation='nearest')
    plt.title(mytitle, size=8)
    plt.gca().set_axis_off()
plt.show()
```

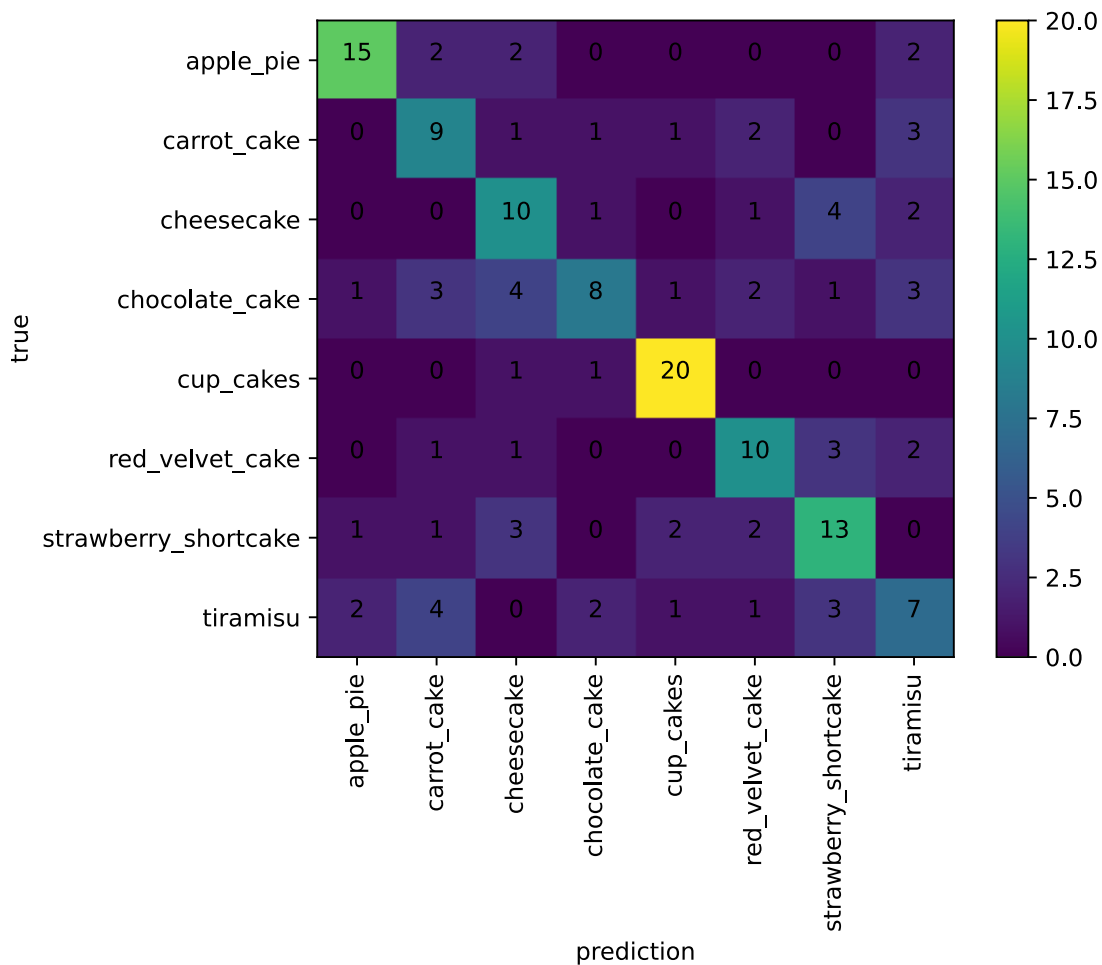


Now look at the confusion matrix

```
[20]: CM = metrics.confusion_matrix(testY, pred_lr)

plt.imshow(CM, interpolation='nearest')
plt.colorbar()

tick_marks = arange(len(le.classes_))
plt.xticks(tick_marks, le.classes_, rotation=90)
plt.yticks(tick_marks, le.classes_)
plt.xlabel('prediction')
plt.ylabel('true')
for i in range(len(le.classes_)):
    for j in range(len(le.classes_)):
        plt.text(j,i,"{}".format(CM[i,j]), horizontalalignment='center')
plt.show()
```



How does the classifier make errors? - **INSERT YOUR ANSWER HERE** - The classifier mainly confuses classes that look alike in color/shape and have similar background settings. In both SVM and LR confusion matrices, “chocolate_cake” and “red_velvet_cake” are sometimes predicted as each other, and “strawberry_shortcake” also gets mixed with other cream-topped cakes. Meanwhile, easier classes like “cup_cakes” (very distinctive shape) are almost always correct. So the errors mostly come from cakes with similar layered/round appearances and toppings, not from the clearly different ones.

2 Transfer learning

Try using a pre-trained network as the backbone for a new network. You can try some of the following: - training an MLP classifier using the 2048 features from ResNet. - using other networks (e.g., InceptionNet) to extract the features. - remember to change the pre-processing step too. - using the a feature map (7x7x2048) and applying a small CNN. - fine-tuning the whole network.

```
[135]: import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

Xtr = trainXf.to(device)
ytr = torch.tensor(trainY, dtype=torch.long).to(device)
Xte = testXf.to(device)
yte = torch.tensor(testY, dtype=torch.long).to(device)

train_ds = TensorDataset(Xtr, ytr)
test_ds = TensorDataset(Xte, yte)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)
```

```
[136]: ### INSERT YOUR CODE HERE ###
# MLP
class MLPClassifier(nn.Module):
    def __init__(self, in_dim=2048, num_classes=8):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_classes)
```

```

    )

    def forward(self, x):
        return self.net(x)

num_classes = 8
model = MLPClassifier(in_dim=2048, num_classes=num_classes).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

epochs = 30
for ep in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        out = model(xb)
        loss = criterion(out, yb)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * xb.size(0)
        _, pred = out.max(1)
        total += yb.size(0)
        correct += (pred == yb).sum().item()

    print(f"Epoch {ep+1}/{epochs}  loss={running_loss/total:.4f}  acc={correct/
↪total:.4f}")

model.eval()
all_preds = []
all_trues = []
with torch.no_grad():
    for xb, yb in test_loader:
        out = model(xb)
        _, pred = out.max(1)
        all_preds.extend(pred.cpu().tolist())
        all_trues.extend(yb.cpu().tolist())

print("Test accuracy:", metrics.accuracy_score(all_trues, all_preds))
print(metrics.classification_report(all_trues, all_preds, target_names=le.
↪classes_))

```

```

Epoch 1/30  loss=1.9737  acc=0.2281
Epoch 2/30  loss=1.4682  acc=0.5266

```

```

Epoch 3/30  loss=1.0202  acc=0.6594
Epoch 4/30  loss=0.6718  acc=0.7750
Epoch 5/30  loss=0.4859  acc=0.8422
Epoch 6/30  loss=0.3235  acc=0.9047
Epoch 7/30  loss=0.2176  acc=0.9516
Epoch 8/30  loss=0.1164  acc=0.9859
Epoch 9/30  loss=0.0853  acc=0.9875
Epoch 10/30 loss=0.0625  acc=0.9922
Epoch 11/30 loss=0.0360  acc=0.9984
Epoch 12/30 loss=0.0327  acc=0.9969
Epoch 13/30 loss=0.0224  acc=1.0000
Epoch 14/30 loss=0.0283  acc=0.9938
Epoch 15/30 loss=0.0281  acc=0.9984
Epoch 16/30 loss=0.0134  acc=1.0000
Epoch 17/30 loss=0.0120  acc=1.0000
Epoch 18/30 loss=0.0075  acc=1.0000
Epoch 19/30 loss=0.0070  acc=1.0000
Epoch 20/30 loss=0.0070  acc=1.0000

Epoch 21/30 loss=0.0071  acc=1.0000
Epoch 22/30 loss=0.0057  acc=1.0000
Epoch 23/30 loss=0.0044  acc=1.0000
Epoch 24/30 loss=0.0043  acc=1.0000
Epoch 25/30 loss=0.0043  acc=1.0000
Epoch 26/30 loss=0.0030  acc=1.0000
Epoch 27/30 loss=0.0029  acc=1.0000
Epoch 28/30 loss=0.0028  acc=1.0000
Epoch 29/30 loss=0.0028  acc=1.0000
Epoch 30/30 loss=0.0056  acc=0.9984

```

Test accuracy: 0.675

	precision	recall	f1-score	support
apple_pie	0.85	0.81	0.83	21
carrot_cake	0.64	0.53	0.58	17
cheesecake	0.52	0.61	0.56	18
chocolate_cake	0.67	0.43	0.53	23
cup_cakes	0.81	1.00	0.90	22
red_velvet_cake	0.67	0.59	0.62	17
strawberry_shortcake	0.67	0.64	0.65	22
tiramisu	0.56	0.75	0.64	20
accuracy			0.68	160
macro avg	0.67	0.67	0.66	160
weighted avg	0.68	0.68	0.67	160

```

[119]: # InceptionNet
import torch.nn.functional as F

```

```

from torchvision.models.feature_extraction import create_feature_extractor
from PIL import Image

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

weights = models.Inception_V3_Weights.IMAGENET1K_V1
preprocess = weights.transforms()
N = len(imgdata)
Xinc = torch.zeros((N, 3, 299, 299))

inception_preprocess = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize(299),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

for i, img in enumerate(imgdata):
    x = np.array(img)
    x = inception_preprocess(x)
    Xinc[i] = x

print(Xinc.shape)

weights = models.Inception_V3_Weights.IMAGENET1K_V1
inception = models.inception_v3(
    weights=weights,
    aux_logits=True
).to(device)
inception.eval()
inception.aux_logits = False

feature_extractor = create_feature_extractor(
    inception,
    return_nodes={"avgpool": "feat"}
).to(device)
feature_extractor.eval()

N = Xinc.size(0)
batch_size = 32
Xf_inc = torch.zeros((N, 2048))

with torch.no_grad():
    for start in range(0, N, batch_size):

```

```

        end = start + batch_size
        if end > N:
            end = N
        xb = Xinc[start:end].to(device)
        out = feature_extractor(xb)
        feats = out["feat"].flatten(1)
        Xf_inc[start:end] = feats.cpu()

print("Inception features shape:", Xf_inc.shape)

(
    trainXf_inc, # inception features, train
    testXf_inc,  # inception features, test
    trainY_inc,  # class labels
    testY_inc,   # class labels
    trainYb_inc, # class one-hot vectors
    testYb_inc,  # class one-hot vectors
    trainXinc,   # processed images for inception
    testXinc,    # processed images for inception
    trainingdata_inc, # raw images
    testingdata_inc  # raw images
) = model_selection.train_test_split(
    Xf_inc, Y, Yb, Xinc, imgdata,
    train_size=0.8, test_size=0.2, random_state=42
)

print(trainXf_inc.shape)
print(testXf_inc.shape)
print(trainY_inc.shape)
print(testY_inc.shape)

# cleanup if you want
del trainingdata_inc, testingdata_inc

# for i, img in enumerate(imgdata):
#     arr = np.array(img)
#     pil_img = Image.fromarray(arr.astype("uint8"), mode="RGB")
#     x = preprocess(pil_img)
#     Xinc[i] = x

# inception = models.inception_v3(
#     weights=weights,
#     aux_logits=True
# ).to(device)
# inception.eval()
# inception.aux_logits = False

```

```

# feat_extractor = create_feature_extractor(
#     inception,
#     return_nodes={"avgpool": "feat"}
# ).to(device)
# feat_extractor.eval()

# batch_size = 32
# Xf_inc = torch.zeros((N, 2048))

# with torch.no_grad():
#     for start in range(0, N, batch_size):
#         end = start + batch_size
#         if end > N:
#             end = N
#         xb = Xinc[start:end].to(device)
#         out = feat_extractor(xb)
#         feats = out["feat"].flatten(1)
#         Xf_inc[start:end] = feats.cpu()

# print("inception features:", Xf_inc.shape)
# print(Xf_inc)

# # Xf_inc: torch.Size([800, 2048])
# X_np = Xf_inc.numpy()
# y_np = Y

# Xtr_np, Xte_np, ytr_np, yte_np = model_selection.train_test_split(
#     X_np, y_np,
#     train_size=0.8,
#     random_state=4487,
#     stratify=y_np
# )

# scaler = StandardScaler()
# Xtr_std = scaler.fit_transform(Xtr_np)
# Xte_std = scaler.transform(Xte_np)

```

```
torch.Size([800, 3, 299, 299])
```

/home/hanzhuo/miniconda3/envs/hz/lib/python3.12/site-packages/torchvision/models/feature_extraction.py:175: UserWarning: NOTE: The nodes obtained by tracing the model in eval mode are a subsequence of those obtained in train mode. When choosing nodes for feature extraction, you may need to specify output nodes for train and eval mode separately.

```
warnings.warn(msg + suggestion_msg)
```

```
Inception features shape: torch.Size([800, 2048])
```

```
torch.Size([640, 2048])
```

```
torch.Size([160, 2048])
(640,)
(160,)
```

```
[134]: Xtr = trainXf_inc.to(device)
ytr = torch.tensor(trainY_inc, dtype=torch.long).to(device)
Xte = testXf_inc.to(device)
yte = torch.tensor(testY_inc, dtype=torch.long).to(device)

train_ds = TensorDataset(Xtr, ytr)
test_ds = TensorDataset(Xte, yte)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)

# Xtr_t = torch.from_numpy(Xtr_std).float().to(device)
# Xte_t = torch.from_numpy(Xte_std).float().to(device)
# ytr_t = torch.from_numpy(ytr_np).long().to(device)
# yte_t = torch.from_numpy(yte_np).long().to(device)

# train_ds = TensorDataset(Xtr_t, ytr_t)
# test_ds = TensorDataset(Xte_t, yte_t)

# train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
# test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)

class NewMLPClassifier(nn.Module):
    def __init__(self, in_dim=2048, num_classes=8):
        super().__init__()
        self.net = nn.Sequential(

            nn.Linear(in_dim, 1024),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(1024, 512),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(512, num_classes)
        )

    def forward(self, x):
        return self.net(x)

num_classes = 8
mlp_inc = NewMLPClassifier(in_dim=2048, num_classes=num_classes).to(device)

criterion = nn.CrossEntropyLoss()
```

```

optimizer = torch.optim.Adam(mlp_inc.parameters(), lr=1e-4)

epochs = 50
for ep in range(epochs):
    mlp_inc.train()
    total = 0
    correct = 0
    running_loss = 0.0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        out = mlp_inc(xb)
        loss = criterion(out, yb)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * xb.size(0)
        _, pred = out.max(1)
        total += yb.size(0)
        correct += (pred == yb).sum().item()

    print(f"Epoch {ep+1}/{epochs}  loss={running_loss/total:.4f}  acc={correct/
↪total:.4f}")

mlp_inc.eval()
all_preds = []
all_trues = []
with torch.no_grad():
    for xb, yb in test_loader:
        out = mlp_inc(xb)
        _, pred = out.max(1)
        all_preds.extend(pred.cpu().tolist())
        all_trues.extend(yb.cpu().tolist())

print("Inception features + MLP  test acc:",
      metrics.accuracy_score(all_trues, all_preds))
print(metrics.classification_report(all_trues, all_preds,
                                    target_names=le.classes_))

```

```

Epoch 1/50  loss=2.0494  acc=0.2203
Epoch 2/50  loss=1.9345  acc=0.3906
Epoch 3/50  loss=1.7588  acc=0.4766
Epoch 4/50  loss=1.5760  acc=0.5234
Epoch 5/50  loss=1.4037  acc=0.5547
Epoch 6/50  loss=1.2745  acc=0.5875
Epoch 7/50  loss=1.1562  acc=0.6391
Epoch 8/50  loss=1.0720  acc=0.6328
Epoch 9/50  loss=0.9981  acc=0.6719
Epoch 10/50 loss=0.9204  acc=0.7094

```

Epoch 11/50	loss=0.8638	acc=0.7031
Epoch 12/50	loss=0.8115	acc=0.7391
Epoch 13/50	loss=0.7595	acc=0.7641
Epoch 14/50	loss=0.6810	acc=0.7984
Epoch 15/50	loss=0.6299	acc=0.8047
Epoch 16/50	loss=0.5708	acc=0.8266
Epoch 17/50	loss=0.5584	acc=0.8344
Epoch 18/50	loss=0.5030	acc=0.8875
Epoch 19/50	loss=0.4779	acc=0.8766
Epoch 20/50	loss=0.4282	acc=0.8953
Epoch 21/50	loss=0.4210	acc=0.8953
Epoch 22/50	loss=0.3627	acc=0.9094
Epoch 23/50	loss=0.3204	acc=0.9344
Epoch 24/50	loss=0.3016	acc=0.9266
Epoch 25/50	loss=0.2776	acc=0.9437
Epoch 26/50	loss=0.2791	acc=0.9437
Epoch 27/50	loss=0.2374	acc=0.9563
Epoch 28/50	loss=0.2131	acc=0.9688
Epoch 29/50	loss=0.1944	acc=0.9797
Epoch 30/50	loss=0.1741	acc=0.9766
Epoch 31/50	loss=0.1609	acc=0.9750
Epoch 32/50	loss=0.1435	acc=0.9875
Epoch 33/50	loss=0.1393	acc=0.9906
Epoch 34/50	loss=0.1263	acc=0.9844
Epoch 35/50	loss=0.1154	acc=0.9922
Epoch 36/50	loss=0.1052	acc=0.9891
Epoch 37/50	loss=0.1053	acc=0.9938
Epoch 38/50	loss=0.0893	acc=0.9953
Epoch 39/50	loss=0.0803	acc=0.9969
Epoch 40/50	loss=0.0773	acc=0.9953
Epoch 41/50	loss=0.0772	acc=0.9953
Epoch 42/50	loss=0.0619	acc=1.0000
Epoch 43/50	loss=0.0616	acc=1.0000
Epoch 44/50	loss=0.0591	acc=0.9969
Epoch 45/50	loss=0.0547	acc=1.0000
Epoch 46/50	loss=0.0430	acc=0.9984
Epoch 47/50	loss=0.0452	acc=0.9984
Epoch 48/50	loss=0.0462	acc=0.9969
Epoch 49/50	loss=0.0385	acc=1.0000
Epoch 50/50	loss=0.0337	acc=1.0000

Inception features + MLP test acc: 0.55

	precision	recall	f1-score	support
apple_pie	0.62	0.68	0.65	22
carrot_cake	0.67	0.35	0.46	23
cheesecake	0.44	0.54	0.48	13
chocolate_cake	0.42	0.36	0.39	22
cup_cakes	0.75	0.88	0.81	24

red_velvet_cake	0.50	0.20	0.29	20
strawberry_shortcake	0.61	0.74	0.67	23
tiramisu	0.32	0.62	0.42	13
accuracy			0.55	160
macro avg	0.54	0.55	0.52	160
weighted avg	0.56	0.55	0.54	160

```
[137]: # CNN
resnet = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2).
    ↪to(device)
resnet.eval()

resnet_featmap = nn.Sequential(*list(resnet.children())[:-2]).to(device)
resnet_featmap.eval()

train_ds_img = TensorDataset(trainXim, torch.tensor(trainY, dtype=torch.long))
test_ds_img = TensorDataset(testXim, torch.tensor(testY, dtype=torch.long))

train_loader_img = DataLoader(train_ds_img, batch_size=32, shuffle=False)
test_loader_img = DataLoader(test_ds_img, batch_size=32, shuffle=False)

train_maps = []
train_labels = []
with torch.no_grad():
    for xb, yb in train_loader_img:
        xb = xb.to(device)
        fm = resnet_featmap(xb)
        train_maps.append(fm.cpu())
        train_labels.append(yb)
train_maps = torch.cat(train_maps, dim=0)
train_labels = torch.cat(train_labels, dim=0)

test_maps = []
test_labels = []
with torch.no_grad():
    for xb, yb in test_loader_img:
        xb = xb.to(device)
        fm = resnet_featmap(xb)
        test_maps.append(fm.cpu())
        test_labels.append(yb)
test_maps = torch.cat(test_maps, dim=0)
test_labels = torch.cat(test_labels, dim=0)

print("train_maps:", train_maps.shape)
print("test_maps:", test_maps.shape)
```

```
train_maps: torch.Size([640, 2048, 7, 7])
test_maps: torch.Size([160, 2048, 7, 7])
```

```
[155]: class SmallCNNHead(nn.Module):
    def __init__(self, in_channels=2048, num_classes=8):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_channels, 512, kernel_size=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),

            nn.Conv2d(512, 512, kernel_size=3, padding=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),

            nn.Conv2d(512, 256, kernel_size=3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(inplace=True),
        )

        self.pool = nn.AdaptiveAvgPool2d((1, 1))

        self.classifier = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x
```

```
[157]: train_ds_maps = TensorDataset(train_maps, train_labels)
test_ds_maps = TensorDataset(test_maps, test_labels)

train_loader = DataLoader(train_ds_maps, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds_maps, batch_size=32, shuffle=False)

model = SmallCNNHead(in_channels=2048, num_classes=len(le.classes_)).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

epochs = 15
for ep in range(epochs):
    model.train()
```

```

total, correct, loss_sum = 0, 0, 0.0
for xb, yb in train_loader:
    xb = xb.to(device)
    yb = yb.to(device)

    optimizer.zero_grad()
    out = model(xb)
    loss = criterion(out, yb)
    loss.backward()
    optimizer.step()

    loss_sum += loss.item() * xb.size(0)
    total += yb.size(0)
    correct += (out.argmax(1) == yb).sum().item()

    print(f"Epoch {ep+1}/{epochs}  loss={loss_sum/total:.4f}  acc={correct/
↪total:.4f}")

model.eval()
all_preds, all_trues = [], []
with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device)
        out = model(xb)
        pred = out.argmax(1).cpu().tolist()
        all_preds.extend(pred)
        all_trues.extend(yb.tolist())

print("Test acc:", metrics.accuracy_score(all_trues, all_preds))
print(metrics.classification_report(all_trues, all_preds, target_names=le.
↪classes_))

```

```

Epoch 1/15  loss=1.4061  acc=0.5047
Epoch 2/15  loss=0.6064  acc=0.8531

Epoch 3/15  loss=0.2572  acc=0.9672
Epoch 4/15  loss=0.1116  acc=0.9922
Epoch 5/15  loss=0.0566  acc=0.9984
Epoch 6/15  loss=0.0299  acc=0.9984
Epoch 7/15  loss=0.0209  acc=1.0000
Epoch 8/15  loss=0.0157  acc=1.0000
Epoch 9/15  loss=0.0097  acc=1.0000
Epoch 10/15  loss=0.0090  acc=1.0000
Epoch 11/15  loss=0.0077  acc=1.0000
Epoch 12/15  loss=0.0071  acc=1.0000
Epoch 13/15  loss=0.0051  acc=1.0000
Epoch 14/15  loss=0.0058  acc=1.0000
Epoch 15/15  loss=0.0050  acc=1.0000

```

Test acc: 0.68125

	precision	recall	f1-score	support
apple_pie	0.95	0.86	0.90	21
carrot_cake	0.50	0.65	0.56	17
cheesecake	0.47	0.50	0.49	18
chocolate_cake	0.79	0.48	0.59	23
cup_cakes	0.83	0.91	0.87	22
red_velvet_cake	0.54	0.76	0.63	17
strawberry_shortcake	0.79	0.68	0.73	22
tiramisu	0.63	0.60	0.62	20
accuracy			0.68	160
macro avg	0.69	0.68	0.67	160
weighted avg	0.70	0.68	0.68	160

```
[161]: # fine-tuning
train_ds = TensorDataset(trainXim, torch.tensor(trainY, dtype=torch.long))
test_ds = TensorDataset(testXim, torch.tensor(testY, dtype=torch.long))

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=32, shuffle=False)

model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2)
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, len(le.classes_))

model = model.to(device)
for p in model.parameters():
    p.requires_grad = True

criterion = nn.CrossEntropyLoss()
params = [
    {"params": [p for name, p in model.named_parameters() if not name.
↳startswith("fc.")], "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3}
]
optimizer = torch.optim.Adam(params, weight_decay=1e-4)
epochs = 15
for ep in range(epochs):
    model.train()
    total, correct, loss_sum = 0, 0, 0.0
    for xb, yb in train_loader:
        xb = xb.to(device)
        yb = yb.to(device)
```

```

optimizer.zero_grad()
out = model(xb)
loss = criterion(out, yb)
loss.backward()
optimizer.step()

loss_sum += loss.item() * xb.size(0)
total += yb.size(0)
correct += (out.argmax(1) == yb).sum().item()

print(f"Epoch {ep+1}/{epochs}  loss={loss_sum/total:.4f}  acc={correct/
↪total:.4f}")

model.eval()
all_preds, all_trues = [], []
with torch.no_grad():
    for xb, yb in test_loader:
        xb = xb.to(device)
        out = model(xb)
        pred = out.argmax(1).cpu().tolist()
        all_preds.extend(pred)
        all_trues.extend(yb.tolist())

print("Fine-tuned test acc:", metrics.accuracy_score(all_trues, all_preds))
print(metrics.classification_report(all_trues, all_preds, target_names=le.
↪classes_))

```

```

Epoch 1/15  loss=1.8966  acc=0.3484
Epoch 2/15  loss=0.7657  acc=0.8625
Epoch 3/15  loss=0.1719  acc=0.9797
Epoch 4/15  loss=0.0388  acc=0.9969
Epoch 5/15  loss=0.0107  acc=1.0000
Epoch 6/15  loss=0.0091  acc=0.9984
Epoch 7/15  loss=0.0045  acc=1.0000
Epoch 8/15  loss=0.0035  acc=1.0000
Epoch 9/15  loss=0.0024  acc=1.0000
Epoch 10/15 loss=0.0023  acc=1.0000
Epoch 11/15 loss=0.0014  acc=1.0000
Epoch 12/15 loss=0.0017  acc=1.0000
Epoch 13/15 loss=0.0011  acc=1.0000
Epoch 14/15 loss=0.0014  acc=1.0000
Epoch 15/15 loss=0.0010  acc=1.0000
Fine-tuned test acc: 0.7

```

	precision	recall	f1-score	support
apple_pie	0.83	0.90	0.86	21
carrot_cake	0.92	0.71	0.80	17
cheesecake	0.43	0.50	0.46	18

chocolate_cake	0.58	0.48	0.52	23
cup_cakes	0.91	0.91	0.91	22
red_velvet_cake	0.78	0.82	0.80	17
strawberry_shortcake	0.68	0.77	0.72	22
tiramisu	0.53	0.50	0.51	20
accuracy			0.70	160
macro avg	0.71	0.70	0.70	160
weighted avg	0.70	0.70	0.70	160

2.1 Analysis

For your best deep learning model, visualize the result and analyze the errors.

```
[164]: # get the class labels
predYcl = le.inverse_transform(all_preds)
testYcl = le.inverse_transform(testY)

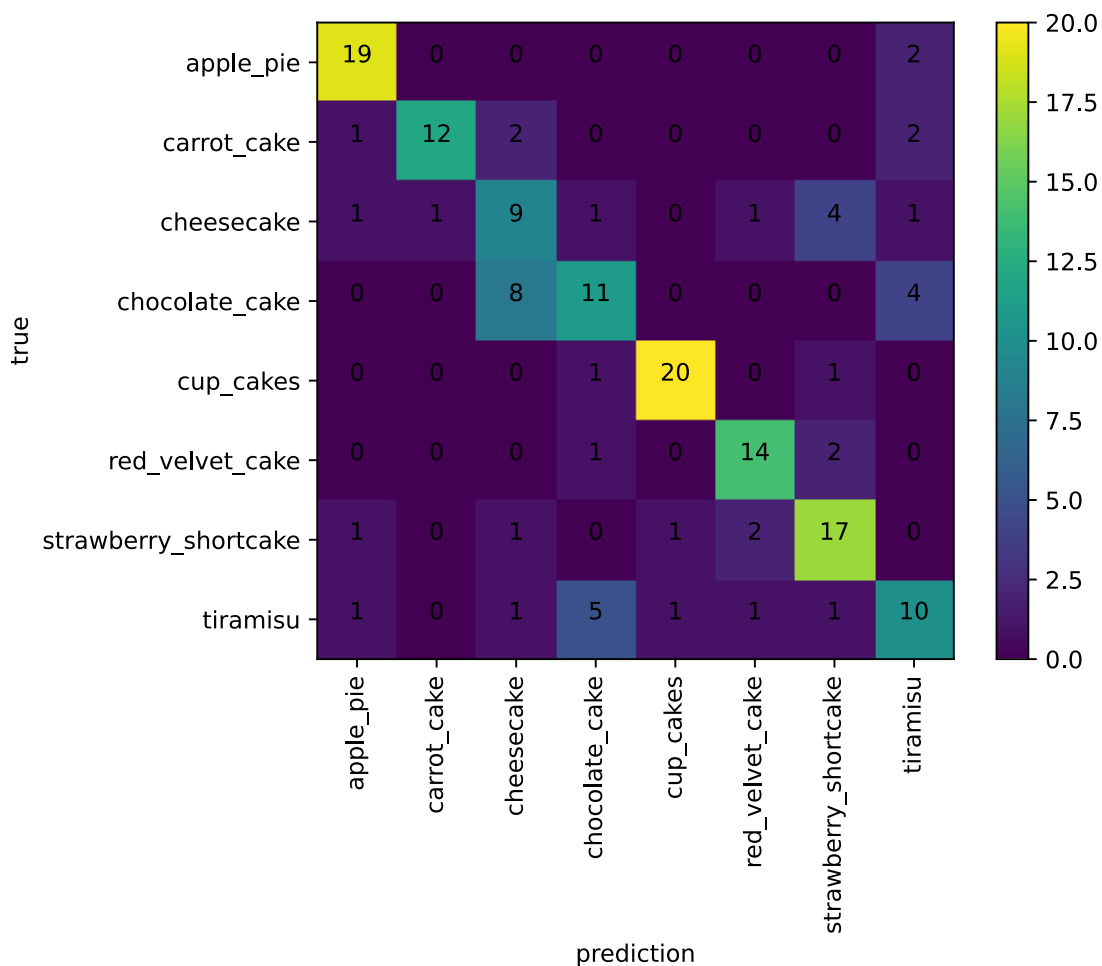
plt.figure(figsize=(10,50))
px = 8
py = int(ceil(len(predYcl)/5))
for i in range(len(predYcl)):
    plt.subplot(py,px,i+1)
    mytitle = predYcl[i] + "\n(" + testYcl[i] + ")"
    plt.imshow(testimgdata[i], interpolation='nearest')
    plt.title(mytitle, size=8)
    plt.gca().set_axis_off()
plt.show()
```



```
[165]: ### INSERT YOUR CODE HERE ###
# CM
CM = metrics.confusion_matrix(testY, all_preds)

plt.imshow(CM, interpolation='nearest')
plt.colorbar()

tick_marks = arange(len(le.classes_))
plt.xticks(tick_marks, le.classes_, rotation=90)
plt.yticks(tick_marks, le.classes_)
plt.xlabel('prediction')
plt.ylabel('true')
for i in range(len(le.classes_)):
    for j in range(len(le.classes_)):
        plt.text(j,i,"{}".format(CM[i,j]), horizontalalignment='center')
plt.show()
```



Where you able to improve the accuracy using deep learning? Analyze the errors from your best model.

- **INSERT YOUR ANSWER HERE**
- Yes, the deep learning models were able to improve accuracy over the earlier, more traditional approaches. Once we used a ResNet backbone and fine-tuned it on the cake dataset, most images were classified correctly and the confusion matrix became mostly diagonal. Classes that have very distinctive shapes or presentations, such as apple_pie and especially cup_cakes, were recognized almost perfectly in the visualization.
- The mistakes that remain follow a clear pattern. The model tends to confuse cakes that look alike in color, topping and slice shape. Cheesecake is sometimes predicted as chocolate_cake or tiramisu because all three can appear as brownish slices on a plate with cream on top. Red_velvet_cake is occasionally taken for strawberry_shortcake or even cheesecake because they share the red-and-white look or layered structure. Carrot_cake also gets pulled toward cheesecake or chocolate_cake when there is cream on top and the background is similar.